O módulo *Port Expander LCD* (*PELCD*) tem como principal objetivo realizar a interface entre o módulo de *Control*, implementado em software, e o periférico LCD.

O módulo *PELCD* é constituído por um bloco único designado por *Serial Receiver*, como representado na Figura 1.

Neste caso, o módulo *Control* é a entidade produtora da informação, enquanto o *LCD* atua como entidade consumidora.

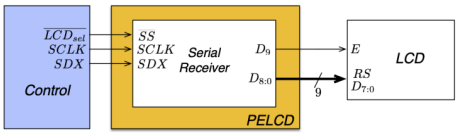

Figura 1: Diagrama de blocos do módulo *Port Expander LCD*

1 Serial Receiver

O bloco *Serial Receiver* é responsável pela receção da informação enviada em série pelo módulo de *Control*, armazenando os bits recebidos e disponibilizando-os posteriormente ao LCD sob a forma de sinais apresentados em série.

De acordo com o diagrama apresentado na Figura 2, este bloco integra dois sub-blocos principais:

- um registo de deslocamento (*Shift Register*);
- um registo de manutenção (*Hold Register*).

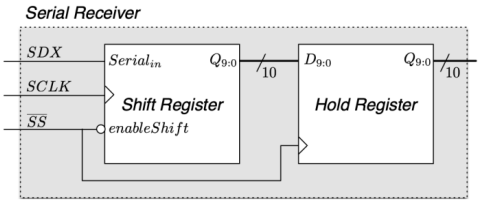

Figura 2: Diagrama de blocos do módulo *Serial Receiver*

A receção da informação é iniciada por uma transição descendente do sinal *LCDsel*, conforme ilustrado na Figura 3, sendo os bits da trama recebidos sequencialmente até se totalizar a informação completa, que, posteriormente, é disponibilizada ao *Hold Register*.

O sinal de *enable* (*E*), incluído na trama recebida, é utilizado para controlar o registo da informação no LCD. Este é gerado pelo módulo de *Control*, sendo alternado entre duas tramas consecutivas.

A informação recebida pelo *Serial Receiver* é organizada sob a forma de uma trama de 10 bits, constituída por um bit de seleção de registo (*RS*), oito bits de dados e um bit de ativação (*E*), sendo os bits transmitidos sequencialmente a partir do bit menos significativo. Esta organização permite, não só distinguir entre comandos (*CMD*) e dados (*DATA*), como também o modo como é controlado o registo da informação no LCD.

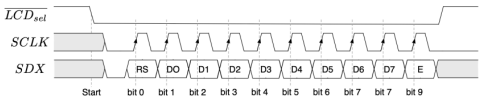

Figura 3: Protocolo de comunicação com o módulo *Port Expander LCD* (*PELCD*)

1.1 Shift Register

O *Shift Register* é responsável pela receção sequencial dos bits de dados provenientes da entrada (*SDX*). A cada transição ascendente do sinal (*SCLK*), um novo bit é introduzido no registo, sendo o conteúdo previamente armazenado deslocado um bit para a direita. Este comportamento resulta da implementação deste registo como um conjunto de *flip-flops* ligados em série.

Deste modo, o registo vai acumulando os bits recebidos até perfazer os 10 bits da trama enviada pelo módulo *Control*, correspondendo à trama completa a disponibilizar ao *Hold Register*.

1.2 Hold Register

O *Hold Register* é responsável pelo armazenamento da palavra completa após a receção dos 10 bits da trama, provenientes do *Shift Register*. Este registo mantém os valores estáveis e guardados, permitindo a sua correta aplicação ao *LCD*.

Este bloco, à semelhança do *Shift Register*, é implementado como um registo de 10 bits constituído por *flip-flops*, que permitem armazenar simultaneamente todos os bits da informação recebida. Apesar desta semelhança, ao contrário do *Shift Register*, neste registo os bits não são deslocados sequencialmente, todos os valores permanecem constantes até à receção de nova informação.

Os bits armazenados neste registo são diretamente aplicados às entradas do LCD, correspondendo ao sinal de seleção de registo (RS), aos oito bits de dados e ao sinal de *enable* (E), responsável pelo registo da informação no *LCD*.

Desta forma, garante-se também que os sinais aplicados ao *LCD* permanecem estáveis durante o processo de escrita.

2 Conclusões

O módulo *Port Expander LCD* permite realizar a interface entre o módulo de *Control* e o *LCD*, assegurando a correta receção e adaptação da informação enviada em série.

A utilização de uma arquitetura baseada num *Serial Receiver*, constituído por um *Shift Register* e um *Hold Register*, permite separar o processo de receção sequencial do armazenamento estável da informação, garantindo a correta aplicação dos dados ao *LCD*.

Através da comunicação série utilizada, é permitida a redução do número de linhas de interligação necessárias, simplificando o sistema sem comprometer a fiabilidade da transmissão. Desta forma, é garantida uma integração eficiente do módulo *Port Expander LCD* na arquitetura global do sistema.

Referências

References

[1] Laboratório de Informática e Computadores - Slides de apoio às aulas, 2025/2026.

[2] Laboratório de Informática e Computadores - Enunciado do Projeto "Ticket Machine", 2025/2026.

[3] Pedro Miguens Matutino, Sérgio André *Guia de Instalação do Quartus Prime Lite Edition 20.1.1.720*, 2022, ISEL.

A VHDL

A.1 PELCD

```
library ieee;
use ieee.std_logic_1164.all;

entity PELCD is
        port(
                LCDsel  : in std_logic;
                SCLK        : in std_logic;
                SDX            : in std_logic;
                rst            : in std_logic;
                D       : out std_logic_vector(8 downto 0);
                D9      : out std_logic
        );
end PELCD;

architecture structural of PELCD is
component SerialReceiver is
        port(
                SS      : in std_logic;
                SCLK    : in std_logic;
                SDX     : in std_logic;
                rst     : in std_logic;
                Q       : out std_logic_vector(9 downto 0)
        );
end component SerialReceiver;

signal RcvOut: std_logic_vector(9 downto 0);
begin
        Rcv: SerialReceiver port map(SS => LCDsel, SCLK => SCLK, SDX => SDX, rst => rst, Q => RcvOut);
        D <= RcvOut(8) & RcvOut(7) & RcvOut(6) & RcvOut(5) & RcvOut(4) & RcvOut(3) & RcvOut(2) & RcvOut(1) & RcvOut(0);
        D9 <= RcvOut(9);
end structural;
```

A.2 PELCD testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity PELCD_tb is
end entity;

architecture PELCD_tb_arch of PELCD_tb is

component PELCD
        port(
                LCDsel, SCLK, SDX, rst  : in std_logic;
                D                       : out std_logic_vector(8 downto 0);
                D9                      : out std_logic
        );
end component;

signal LCDsel_TB, SDX_TB, rst_TB, D9_TB : std_logic;
signal SClk_TB                          : std_logic := '1';
signal D_TB                             : std_logic_vector(8 downto 0);
constant SClk_period                    : time := 10000 ps;
constant SClk_half_period               : time := SClk_period / 2;

begin
        SClk_TB <= not(SClk_TB) after SClk_half_period;
        UUT: PELCD port map(
                LCDsel  => LCDsel_TB,
                SCLK    => SClk_TB,
                SDX     => SDX_TB,
                rst     => rst_TB,
                D       => D_TB,
                D9      => D9_TB
        );
stimulus: process
begin
        LCDsel_TB   <= '1';
        SDX_TB      <= '1';
        rst_TB      <= '1';
        wait for SClk_period;
        rst_TB      <= '0';
        LCDsel_TB   <= '0';
        wait for SClk_period * 11;
        LCDsel_TB   <= '1';
        wait for SClk_period * 2;
end process;
end architecture;
```

A.3 RG10

```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY RG10 IS
        PORT(
                D               : in std_logic_vector(9 downto 0);
                CLK,EN,Reset    : in std_logic;
                Q               : out STD_LOGIC_vector(9 downto 0)
        );
END RG10;

ARCHITECTURE logicFunction OF RG10 IS
BEGIN
        FDD0: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(0), Q=>Q(0), SET =>'0', EN=>EN);
        FDD1: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(1), Q=>Q(1), SET =>'0', EN=>EN);
        FDD2: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(2), Q=>Q(2), SET =>'0', EN=>EN);
        FDD3: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(3), Q=>Q(3), SET =>'0', EN=>EN);
        FDD4: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(4), Q=>Q(4), SET =>'0', EN=>EN);
        FDD5: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(5), Q=>Q(5), SET =>'0', EN=>EN);
        FDD6: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(6), Q=>Q(6), SET =>'0', EN=>EN);
        FDD7: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(7), Q=>Q(7), SET =>'0', EN=>EN);
        FDD8: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(8), Q=>Q(8), SET =>'0', EN=>EN);
        FDD9: entity work.FFD port
                map (CLK=>CLK, RESET =>Reset,D=>D(9), Q=>Q(9), SET =>'0', EN=>EN);
END LogicFunction;
```

A.4 Serial Receiver

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialReceiver is
        port(
                SS, SCLK, SDX, rst  : in std_logic;
                Q                   : out std_logic_vector(9 downto 0)
        );
end SerialReceiver;

architecture structural of SerialReceiver is

component ShiftReg is
        port(
                SerialIn    : in std_logic;
                SCLK        : in std_logic;
                enableShift : in std_logic;
                rst         : in std_logic;
                Q           : out std_logic_vector(9 downto 0)
        );
end component ShiftReg;

component RG10 is
        PORT(
                D               : in  std_logic_vector(9 downto 0);
                Q               : out STD_LOGIC_vector(9 downto 0);
                CLK,EN,Reset    : in STD_LOGIC
                );
end component RG10;

signal toReg : std_logic_vector(9 downto 0);
signal nSS   : std_logic;

begin
        nSS <= not(SS);
        ShiReg:        ShiftReg    port map(SerialIn => SDX, SCLK => SCLK, enableShift => nSS,
                rst => rst, Q => toReg);
        HldReg:        RG10        port map(D => toReg, Q => Q, CLK => SS, EN => '1',
                Reset => rst);
end structural;
```

A.5 Serial Receiver testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity SerialReceiver_tb is
end entity;

architecture SerialReceiver_tb_arch of SerialReceiver_tb is

component SerialReceiver
        port(
                SS, SCLK, SDX, rst  : in std_logic;
                Q                   : out std_logic_vector(9 downto 0)
        );
end component;

signal SS_TB, SDX_TB, rst_TB    : std_logic;
signal SClk_TB                  : std_logic := '1';
signal Q_TB                     : std_logic_vector(9 downto 0);
constant SClk_period            : time := 10000 ps;
constant SClk_half_period       : time := SClk_period / 2;

begin
        SClk_TB <= not(SClk_TB) after SClk_half_period;
        UUT: SerialReceiver port map(
                SS      => SS_TB,
                SCLK    => SCLK_TB,
                SDX     => SDX_TB,
                rst     => rst_TB,
                Q       => Q_TB
        );
stimulus: process
begin
        SS_TB   <= '1';
        SDX_TB  <= '1';
        rst_TB  <= '1';
        wait for SClk_period;
        rst_TB  <= '0';
        SS_TB   <= '0';
        wait for SClk_period * 11;
        SS_TB   <= '1';
        wait for SClk_period;
end process;
end architecture;
```

A.6 Shift Register

```
library ieee;
use ieee.std_logic_1164.all;

entity ShiftReg is
        port(
                SerialIn    : in std_logic;
                SCLK        : in std_logic;
                enableShift : in std_logic;
                rst         : in std_logic;
                Q           : out std_logic_vector(9 downto 0)
        );
end ShiftReg;

architecture structural of ShiftReg is

component FFD is
        port(
                CLK, RESET, SET, D, EN  : in std_logic;
                Q                       : out std_logic
        );
end component FFD;

signal nextFFD: std_logic_vector(9 downto 0);

begin
        FF9: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => SerialIn  ,
                Q => nextFFD(9));
        FF8: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(9),
                Q => nextFFD(8));
        FF7: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(8),
                Q => nextFFD(7));
        FF6: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(7),
                Q => nextFFD(6));
        FF5: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(6),
                Q => nextFFD(5));
        FF4: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(5),
                Q => nextFFD(4));
        FF3: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(4),
                Q => nextFFD(3));
        FF2: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(3),
                Q => nextFFD(2));
        FF1: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(2),
                Q => nextFFD(1));
        FF0: FFD port map(CLK => SCLK, RESET => rst, SET => '0', EN => enableShift, D => nextFFD(1),
                Q => nextFFD(0));
        Q <= nextFFD;
end structural;
```

A.7 Shift Register testbench

```
library ieee;
use ieee.std_logic_1164.all;

entity ShiftReg_tb is
end entity;

architecture ShiftReg_tb_arch of ShiftReg_tb is

component ShiftReg
        port(
                SerialIn, SCLK, enableShift, rst    : in std_logic;
                Q                                   : out std_logic_vector(9 downto 0)
        );
end component;

signal SerialIn_TB, enableShift_TB, rst_TB  : std_logic;
signal SClk_TB                              : std_logic := '1';
signal Q_TB                                 : std_logic_vector(9 downto 0);
constant SClk_period                        : time := 10000 ps;
constant SClk_half_period                   : time := SClk_period / 2;

begin
        SClk_TB <= not(SClk_TB) after SClk_half_period;
        UUT: ShiftReg port map(
                SerialIn    => SerialIn_TB,
                SCLK        => SCLK_TB,
                enableShift => enableShift_TB,
                rst         => rst_TB,
                Q           => Q_TB
        );
stimulus: process
begin
        SerialIn_TB     <= '1';
        enableShift_TB  <= '0';
        rst_TB          <= '1';
        wait for SClk_period;
        rst_TB          <= '0';
        enableShift_TB  <= '1';
        wait for SClk_period * 11;
end process;
end architecture;
```

B *Kotlin*

B.1 LCD

Algoritmo 1: LCD - Liquid Cristal Display

```
// Escreve no LCD usando a interface a 8 bits.
object LCD {
    // Dimensao do display.
    const val LINES = 2
    const val COLS = 16

    // Escreve um byte de comando / dados no LCD em serie
    fun writeByteSerial(rs: Boolean, data: Int) {
        val rsBit = if (rs) 1 else 0
        val notRsBit = if (rs) 0 else 1

        val firstFrame = (1 shl 9) or (data shl 1) or rsBit
        val secondFrame = (0 shl 9) or (data shl 1) or rsBit

        SerialEmitter.send(SerialEmitter.Peripheral.LCD, firstFrame)
        SerialEmitter.send(SerialEmitter.Peripheral.LCD, secondFrame)
    }

    // Escreve um byte de comando / dados no LCD
    fun writeByte (rs: Boolean, data: Int) {
        writeByteSerial(rs, data)
        Time.sleep(2)
    }

    // Escreve um comando no LCD
    fun writeCMD (data : Int ) {
        writeByte(false, data)
    }

    // Escreve um dado no LCD
    fun writeDATA (data : Int ) {
        writeByte(true, data)
    }

    // Envia a sequencia de iniciacao para comunicacao a 8 bits.
    fun init() {
        LCD.clear()
        LCD.writeCMD(0b00110000)
        LCD.writeCMD(0b00110000)
        LCD.writeCMD(0b00110000)

        LCD.writeCMD(0b00111000)//+10==2  2 linhas N;
        LCD.writeCMD(0b00001000)
        LCD.writeCMD(0b00000001)
        LCD.writeCMD(0b00000110)
        LCD.writeCMD(0b00001111)
    }

    // Escreve um carater na posicao corrente.
    fun write (c : Char ) {
        writeDATA(c.code)
    }

    // Escreve uma string na posicao corrente.
    fun write (text : String ) {
        for (i in text){
            write(i)
        }
    }

    // Envia comando para posicionar cursor ('line': 0..LINES-1, 'column': 0..COLS-1)
    fun cursor(line: Int, column : Int ) {
        val address = if (line == 0) column else 0x40 + column
        writeCMD(0x80 or address)
    }
```

```
    // Envia comando para limpar o ecra e posicionar o cursor em (0,0)
    fun clear() {
        writeCMD(0x01)
    }
}
```

B.2 SerialEmitter

Algoritmo 2: Serial Emitter

```
// Envia tramas para os diferentes modulos Serial Receiver.
object SerialEmitter {
    enum class Peripheral {LCD, TICKET}

    // Inicia a classe
    fun init() {}

    // Envia uma trama para o SerialReceiver identificado o periferico de destino em 'addr', os bits de dados
    // 'size' o numero de bits a enviar.
    fun send(addr: Peripheral, data: Int) {
        val sel = LCDsel

        val interval = 1L

        HAL.setBits(sel)
        HAL.clrBits(sCLK)

        HAL.clrBits(sel)

        for (i in 0.. 9) {
            val bit = (data shr i) and 1

            if (bit == 1) HAL.setBits(sdx)
            else HAL.clrBits(sdx)

            Time.sleep(interval)

            HAL.setBits(sCLK)
            Time.sleep(interval)
            HAL.clrBits(sCLK)
        }

        // End transmission
        HAL.setBits(sel)
    }

}
```